

Министерство образования и науки РФ
Федеральное государственное автономное образовательное учреждение высшего
образования
«Омский государственный технический университет»

Факультет (институт) Информационных технологий и компьютерных систем

Кафедра Прикладная математика и фундаментальная информатика

Расчетно-графическая работа

по дисциплине Алгоритмы и анализ сложности

на тему Анализ сложности работы алгоритмов

Пояснительная записка

Шифр проекта 020-РГР-02.03.02-№ 37-ПЗ

Студента

фамилия, имя, отчество полностью

Курс 3

Группа ФИТ-231

Направление (специальность)

02.03.02

Фундаментальная информатика и информационные технологии

код, наименование

Руководитель

ученая степень, звание

фамилия, инициалы

Выполнил

дата, подпись студента

Работа защищена с количеством баллов

дата, подпись руководителя

Омск 2026

Содержание

Задание 1	3
Задание 2	6

Задание 1

Необходимо вычислить n -й член последовательности Фиббоначи двумя методами:

- 1) Рекурсивным;
- 2) Итерационным.

Код реализации:

```
import time
import matplotlib.pyplot as plt

def fibonacci_recursive(n):
    if n <= 1:
        return n
    return fibonacci_recursive(n-1) + fibonacci_recursive(n-2)

def fibonacci_iterative(n):
    if n <= 1:
        return n

    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

def measure_time(func, n):
    start_time = time.perf_counter()
    result = func(n)
    end_time = time.perf_counter()
    return result, (end_time - start_time) * 1000

max_n = 40
recursive_times = []
iterative_times = []
n_values = list(range(1, max_n + 1))

print("-" * 60)
print(f"{'n':>3} | {'Рекурсивно (мс)':>15} | {'Итерационно (мс)':>15} | {'Значение':>15}")
print("-" * 60)

for n in n_values:
    value, iter_time = measure_time(fibonacci_iterative, n)
    iterative_times.append(iter_time)

    _, rec_time = measure_time(fibonacci_recursive, n)
    recursive_times.append(rec_time)

    print(f"{'n':>3} | {rec_time:15.6f} | {iter_time:15.6f} | {value:15}")

print("-" * 60)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
```

```

ax1.plot(n_values, iterative_times, 'g-', linewidth=2, label='Итерационный')
ax1.set_xlabel('Номер элемента (n)', fontsize=12)
ax1.set_ylabel('Время выполнения (мс)', fontsize=12)
ax1.set_title('Итерационное вычисление', fontsize=14, fontweight='bold')
ax1.grid(True, alpha=0.3)
ax1.legend()

ax2.plot(n_values, recursive_times, 'r-', linewidth=2, label='Рекурсивный')
ax2.set_xlabel('Номер элемента (n)', fontsize=12)
ax2.set_ylabel('Время выполнения (мс)', fontsize=12)
ax2.set_title('Рекурсивное вычисление', fontsize=14, fontweight='bold')
ax2.grid(True, alpha=0.3)
ax2.legend()

plt.suptitle('Сравнение времени вычисления чисел Фибоначчи', fontsize=16,
fontweight='bold', y=1.02)

plt.tight_layout()
plt.savefig('fibonacci_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

Пример выполнения:

n	Рекурсивно (мс)	Итерационно (мс)	Значение
1	0.000800	0.001200	1
2	0.001400	0.003400	1
3	0.002700	0.001100	2
4	0.002400	0.002200	3
5	0.002200	0.002200	5
6	0.002900	0.001400	8
7	0.004200	0.001000	13
8	0.005400	0.000900	21
9	0.010500	0.000900	34
10	0.017500	0.003900	55
11	0.025100	0.001500	89
12	0.039700	0.001400	144
13	0.062700	0.001500	233
14	0.108900	0.001600	377
15	0.165300	0.001900	610
16	0.248100	0.002000	987
17	0.540600	0.005800	1597
18	1.032000	0.005200	2584
19	1.170500	0.005400	4181
20	1.639300	0.005800	6765
21	1.965900	0.004600	10946
22	4.556600	0.002500	17711
23	8.981100	0.008800	28657
24	12.681000	0.009700	46368
25	17.134100	0.010500	75025
26	30.882300	0.023200	121393
27	44.252100	0.007500	196418

Рисунок 1 – Пример выполнения первого задания

Графики времени выполнения и количества итераций для методов.

Сравнение времени вычисления чисел Фибоначчи

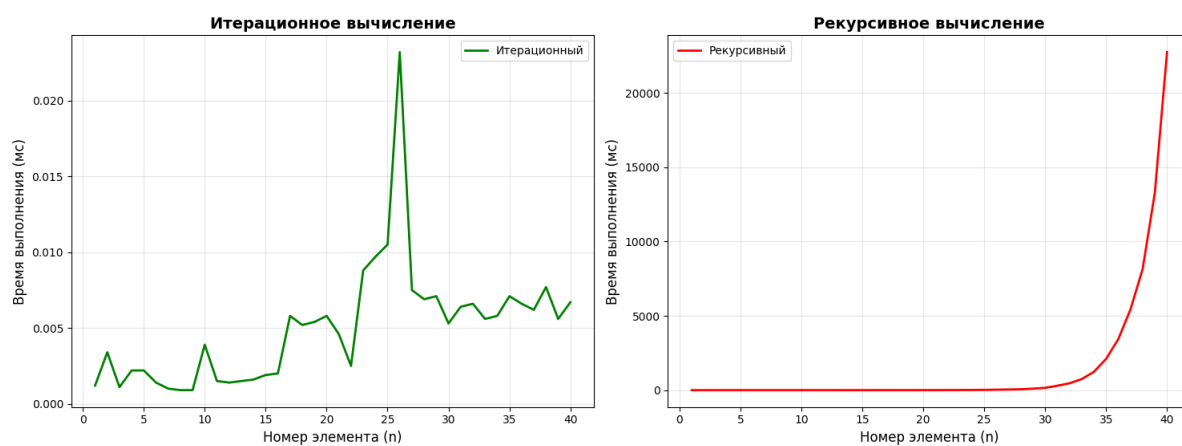


Рисунок 2 – Графики времени выполнения и количества итераций первого задания

Задание 2

Необходимо сравнить три алгоритма сортировки:

- 1) Гномий;
- 2) «Пузырек»;
- 3) Вставками.

Код реализации:

```
import random
import time
import matplotlib.pyplot as plt
import numpy as np

def bubble_sort(arr):
    n = len(arr)
    comparisons = 0
    swaps = 0
    arr_copy = arr.copy()

    for i in range(n - 1):
        for j in range(n - i - 1):
            comparisons += 1
            if arr_copy[j] > arr_copy[j + 1]:
                arr_copy[j], arr_copy[j + 1] = arr_copy[j + 1], arr_copy[j]
                swaps += 1

    return arr_copy, comparisons, swaps

def gnome_sort(arr):
    n = len(arr)
    comparisons = 0
    swaps = 0
    arr_copy = arr.copy()

    index = 0
    while index < n:
        if index == 0:
            index += 1
        comparisons += 1
        if arr_copy[index] >= arr_copy[index - 1]:
            index += 1
        else:
            arr_copy[index], arr_copy[index - 1] = arr_copy[index - 1],
arr_copy[index]
            swaps += 1
            index -= 1

    return arr_copy, comparisons, swaps

def insertion_sort(arr):
    n = len(arr)
    comparisons = 0
    swaps = 0
```

```

arr_copy = arr.copy()

for i in range(1, n):
    key = arr_copy[i]
    j = i - 1
    while j >= 0 and arr_copy[j] > key:
        comparisons += 1
        arr_copy[j + 1] = arr_copy[j]
        swaps += 1
        j -= 1
    if j >= 0:
        comparisons += 1
    arr_copy[j + 1] = key
    if j + 1 != i:
        swaps += 1

return arr_copy, comparisons, swaps

def measure_sort_time(sort_func, arr):
    start_time = time.perf_counter()
    sorted_arr, comparisons, swaps = sort_func(arr)
    end_time = time.perf_counter()

    execution_time = (end_time - start_time) * 1000
    return execution_time, comparisons, swaps

def generate_random_array(size, min_val=1, max_val=1000):
    return [random.randint(min_val, max_val) for _ in range(size)]

def measure_sort_time(sort_func, arr):
    start_time = time.perf_counter()
    sorted_arr, comparisons, swaps = sort_func(arr)
    end_time = time.perf_counter()

    execution_time = (end_time - start_time) * 1000
    return execution_time, comparisons, swaps

def generate_random_array(size, min_val, max_val):
    return [random.randint(min_val, max_val) for _ in range(size)]

sizes = [10, 50, 100, 200, 500, 1000, 2000, 3000, 4000, 5000]

MIN_VALUE = 1
MAX_VALUE = 10000

bubble_times = []
gnome_times = []
insertion_times = []

bubble_comparisons = []
gnome_comparisons = []
insertion_comparisons = []

bubble_swaps = []
gnome_swaps = []
insertion_swaps = []

print("=" * 100)
print(f"{'Размер':^10} | {'Пузырек (мс)':^15} | {'Гномья (мс)':^15} |  

{'Вставки (мс)':^15} | {'Сравнения (П/Г/В)':^35}")

```

```

print("=" * 100)

for size in sizes:
    arr = generate_random_array(size, MIN_VALUE, MAX_VALUE)

    bubble_time, bubble_comp, bubble_swap = measure_sort_time(bubble_sort,
arr)
    bubble_times.append(bubble_time)
    bubble_comparisons.append(bubble_comp)
    bubble_swaps.append(bubble_swap)

    gnome_time, gnome_comp, gnome_swap = measure_sort_time(gnome_sort, arr)
    gnome_times.append(gnome_time)
    gnome_comparisons.append(gnome_comp)
    gnome_swaps.append(gnome_swap)

    insertion_time, insertion_comp, insertion_swap =
measure_sort_time(insertion_sort, arr)
    insertion_times.append(insertion_time)
    insertion_comparisons.append(insertion_comp)
    insertion_swaps.append(insertion_swap)

    print(f"{size:^10} | {bubble_time:^15.3f} | {gnome_time:^15.3f} |
{insertion_time:^15.3f} | "
          f"{bubble_comp:^10}|{gnome_comp:^10}|{insertion_comp:^10}")

print("=" * 100)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 7))

ax1.plot(sizes, bubble_times, 'ro-', linewidth=2, markersize=6,
label='Пузырек')
ax1.plot(sizes, gnome_times, 'gs-', linewidth=2, markersize=6,
label='Гномья')
ax1.plot(sizes, insertion_times, 'b^-', linewidth=2, markersize=6,
label='Вставки')

ax1.set_xlabel('Размер входных данных (n)', fontsize=12)
ax1.set_ylabel('Время выполнения (мс)', fontsize=12)
ax1.set_title('Сравнение времени выполнения сортировок', fontsize=14,
fontweight='bold')
ax1.grid(True, alpha=0.3, linestyle='--')
ax1.legend(fontsize=11)

ax2.plot(sizes, bubble_comparisons, 'ro-', linewidth=2, markersize=6,
label='Пузырек')
ax2.plot(sizes, gnome_comparisons, 'gs-', linewidth=2, markersize=6,
label='Гномья')
ax2.plot(sizes, insertion_comparisons, 'b^-', linewidth=2, markersize=6,
label='Вставки')

ax2.set_xlabel('Размер входных данных (n)', fontsize=12)
ax2.set_ylabel('Количество сравнений', fontsize=12)
ax2.set_title('Показатель качества сортировки (сравнения)', fontsize=14,
fontweight='bold')
ax2.grid(True, alpha=0.3, linestyle='--')
ax2.legend(fontsize=11)

```



```
plt.suptitle('Сравнительный анализ алгоритмов сортировки', fontsize=16,
fontweight='bold', y=1.02)

plt.tight_layout()
plt.savefig('sorting_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

print(f"{'Алгоритм':<15} | {'Среднее время (мс)':<20} | {'Среднее  
сравнений':<20} | {'Среднее обменов':<20}")
print("-" * 100)
print(f"{'Пузырек':<15} | {np.mean(bubble_times):<20.3f} |  
{np.mean(bubble_comparisons):<20.0f} | {np.mean(bubble_swaps):<20.0f}")
print(f"{'Гномья':<15} | {np.mean(gnome_times):<20.3f} |  
{np.mean(gnome_comparisons):<20.0f} | {np.mean(gnome_swaps):<20.0f}")
print(f"{'Вставки':<15} | {np.mean(insertion_times):<20.3f} |  
{np.mean(insertion_comparisons):<20.0f} |  
{np.mean(insertion_swaps):<20.0f}")
```

Пример выполнения:

Размер	Пузырек (мс)	Гномья (мс)	Вставки (мс)	Сравнения (П/Г/В)		
10	0.009	0.005	0.004	45	40	24
50	0.082	0.097	0.044	1225	1380	713
100	0.293	0.459	0.182	4950	5014	2554
200	1.222	4.612	0.661	19900	20136	10164
500	8.870	11.149	4.582	124750	129459	64975
1000	38.984	40.550	18.202	499500	503200	252097
2000	147.544	169.679	69.782	1999000	1984071	993030
3000	331.611	370.528	166.417	4498500	4404874	2203932
4000	611.456	671.375	281.141	7998000	7991318	3997654
5000	901.497	1020.975	439.455	12497500	12462034	6233513

Рисунок 3 – Пример выполнения второго задания

Графики времени выполнения и количества сравнений для алгоритмов.

Сравнительный анализ алгоритмов сортировки

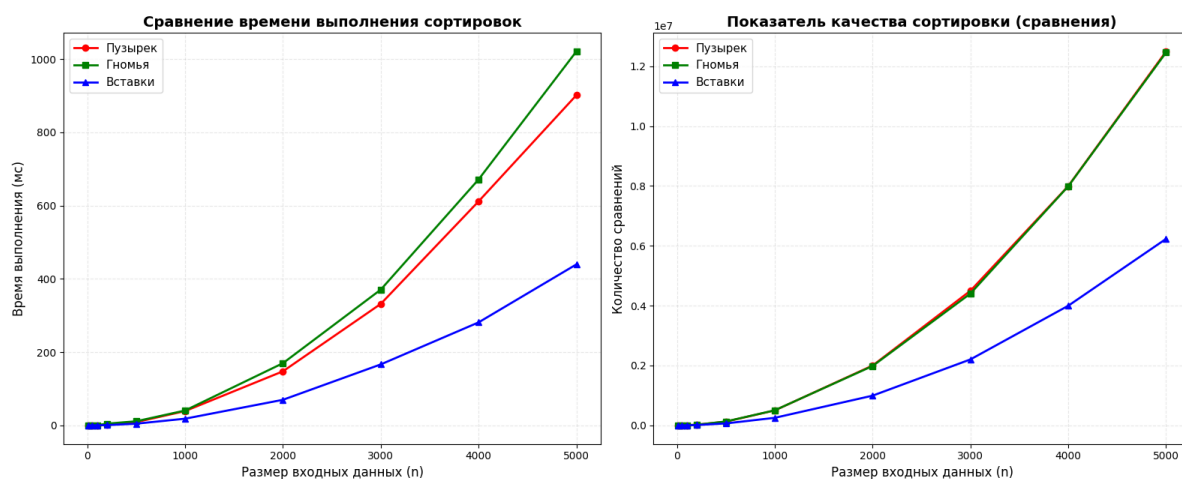


Рисунок 4 – Графики времени выполнения и количества сравнений для алгоритмов второго задания